PROJEKTDOKUMENTATION

Manga DB

Spring Boot | React | JWT | MySQL | Docker

Modul 223 — Projekt

Fabian Imhasly

Elias Kaiser

223_Projekt_fresh

2025

Inhaltsverzeichnis

Inhaltsverzeichnis

Inhaltsverzeichnis 2
1. Projektüberblick 4
1.1 Ausgangslage 4
1.2 Ziel des Projekts 4
1.3 Zielgruppe 4
1.4 Projektumfang 5
2. Drei User Stories mit Akzeptanzkriterien 5
User Story 1: Registrierung und Login 5
User Story 2: Eigene Manga-Status verwalten 5
User Story 3: Stammdaten als Admin pflegen 5
3. Backend-Dokumentation 6
3.1 Verwendete Technologien 6
2.State-Mangagment-Flow 6
3.2 Backend-Architektur 7
3.3 Architekturdiagramm 8
3.4 Wichtige Endpunkte 9
3.5 Transaktionen im Backend 9
4. Datenmodell und Diagramme 10
4.1 ERD-Diagramm 10
4.2 Beschreibung der Beziehungen 11
4.3 Seed-Daten 11
5. JWT-Authentifizierung und Sicherheitskonzept 11
5.1 Sicherheitsziel 11
5.2 Rollenmodell 11
5.3 JWT-Flow 11
5.4 JWT-Ablaufdiagramm 12
5.5 Schutzmechanismen 13
5.6 Besitzlogik bei UserManga 13
5.7 CORS und Frontend-Integration 13
6. Frontend-Dokumentation 13
6.1 Verwendete Technologien 13
6.2 Frontend-Architektur 14
6.3 Komponenten-Hierarchie 14
6.4 State-Management und AuthContext 15
6.5 API-Integration(Service Layer) 15

..... 15
6.6 Rollenabhangige Darstellung im Frontend 17
7. Reading Sessions 17
7.1 Idee 17
7.2 Funktionslogik 17
7.3 Nutzen 17
8. Leaderboard 17
8.1 Idee 17
8.2 Bewertungslogik 17
8.3 Leaderboard-Endpunkte 18
8.4 Nutzen 18
9. Testdokumentation 18
9.1 Backend-Tests 18
Test 1: AuthControllerIntegrationTest 18
Test 2: UserMangaControllerSecurityTest 18
Test 3: LeaderboardControllerIntegrationTest 18
Test 4: MangaControllerSecurityTest 18
9.2 Frontend-Tests 19
Test 1: authUtils.test.js 19
Test 2: mangaUtils.test.js 19
Test 3: leaderboardUtils.test.js 19
9.3 Testprotokoll 19
10. Git und Entwicklungsprozess 20
10.1 Verwendete Remote Branches 20
10.2 Nutzen der Branch-Strategie 20
10.3 Beispielhafte Commit-Gruppen 20
10.4 Warum Git für die LB wichtig ist 20
11. Start und Betrieb des Projekts 20
11.1 Datenbank starten 20
11.2 Backend starten 20
11.3 Frontend starten 21
11.4 Test-Logins 21

1. Projektüberblick

1.1 Ausgangslage

Im Projekt 223 wurde eine webbasierte Manga-Verwaltungsanwendung mit. Die Anwendung besteht aus einem Spring-Boot-Backend und einem React-Frontend. Ziel ist es, Mangas, Kategorien und Verlage zentral zu verwalten, Benutzer mit Rollen zu authentifizieren und pro Benutzer einen eigenen Lesestatus zu führen.

Die Anwendung lost ein typisches Organisationsproblem: Wer viele Manga-Reihen sammelt oder liest, verliert schnell den Überblick über Titel, Bände, Status und Bewertungen. Die Manga DB bietet dafür eine einfache Oberflache und eine abgesicherte REST-API und dien t als eine Librarysammlung für Mangas wo User ihre Lesefortschritte sehen können und mit

1.2 Ziel des Projekts

Die Hauptziele des Projekts sind:

- Benutzer können sich registrieren und anmelden.
- Nur authentifizierte Benutzer können geschützte Inhalte sehen.
- Rollen (USER, ADMIN) steuern, welche Funktionen verfügbar sind.
- Jeder Benutzer kann seine eigenen Manga-Status pflegen.
- Administratoren können Stammdaten wie Mangas, Kategorien, Verlage und Benutzer verwalten.
- Ein Leaderboard zeigt, welcher Benutzer die meisten Mangas gelesen hat in einem Dashboard mit Rangliste

1.3 Zielgruppe

Die Anwendung richtet sich an:

- Manga-Leserinnen und Manga-Leser, die ihre Sammlung oder ihren Fortschritt verwalten wollen

1.4 Projektumfang

Das Projekt umfasst:

- Backend mit REST-API, Security, JWT und MySQL
- Frontend mit React, Routing, AuthContext und geschützten Seiten
- Rollen- und Besitzlogik
- automatisierte Backend- und Frontend-Tests
- Docker für die lokale Datenbank
- User Stories

2. Drei User Stories mit Akzeptanzkriterien

User Story 1: Registrierung und Login

Als neuer Benutzer möchte ich mich registrieren und danach anmelden können, damit ich die geschützten Funktionen der Manga DB nutzen kann.

Akzeptanzkriterien

- Ein neuer Benutzer kann sich mit Username, E-Mail und Passwort registrieren.
- Bei erfolgreicher Registrierung wird der Benutzer mit der Rolle USER angelegt.
- Ein bestehender Benutzer kann sich mit Username oder E-Mail und Passwort anmelden.
- Nach erfolgreichem Login erhalt das Frontend einen JWT-Token.
- Der Token wird gespeichert und bei späteren API-Requests automatisch mitgesendet.
- Nicht authentifizierte Benutzer werden auf die Login-Seite umgeleitet.

User Story 2: Eigene Manga-Status verwalten

Als angemeldeter Benutzer möchte ich für meine Mangas einen eigenen Status setzen, damit ich weiß, was ich gelesen habe, gerade lese oder später lesen möchte.

Akzeptanzkriterien

- Ein Benutzer kann für einen Manga einen Status-Eintrag anlegen.
- Ein Benutzer kann nur die eigenen Status-Einträge sehen.
- Ein Benutzer kann die eigenen Einträge bearbeiten und löschen.
- Ein Benutzer kann keine Status für andere Benutzer anlegen oder andern.
- Pro Benutzer und Manga darf nur ein Status-Eintrag existieren.

User Story 3: Stammdaten als Admin pflegen

Als Administrator möchte ich Mangas, Kategorien, Verlage und Benutzer verwalten können, damit die Datenbasis der Anwendung aktuell und korrekt bleibt.

Akzeptanzkriterien

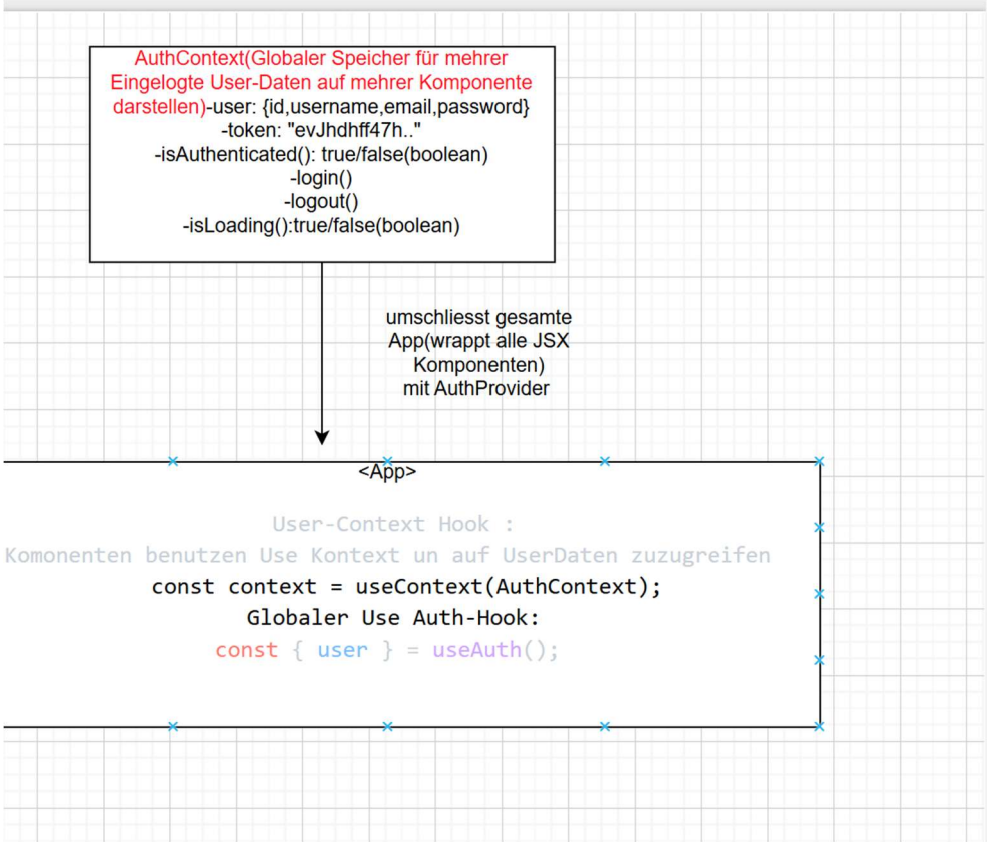
- Nur Benutzer mit Rolle ADMIN sehen die Admin-Funktionen.
- Ein Admin kann neue Mangas anlegen und bestehende bearbeiten oder löschen.
- Ein Admin kann Kategorien und Verlage pflegen.
- Ein Admin kann Benutzer anzeigen und verwalten.
- Ein normaler Benutzer darf diese Funktionen nicht ausführen.

3. Backend-Dokumentation

3.1 Verwendete Technologien

Technologie	Version / Stand	Verwendung
Java	17	Programmiersprache im Backend
Spring Boot	3.2.0	Basis-Framework für die REST-API
Spring Security	Bestandteil von Spring Boot	Absicherung der Endpunkte
JJWT	0.12.6	Erzeugen und Prüfen von JWT-Tokens via JwtService
Spring Data JPA	Bestandteil von Spring Boot	Datenzugriff auf relationale Tabellen
Hibernate	über JPA	ORM zwischen Java-Klassen und MySQL
MySQL	8.0 im Docker-Container	Persistente relationale Datenbank
Maven	über pom.xml	Build- und Dependency-Management
H2	Test-Dependency	Datenbank für Integrationstests

2.State-Mangagment-Flow



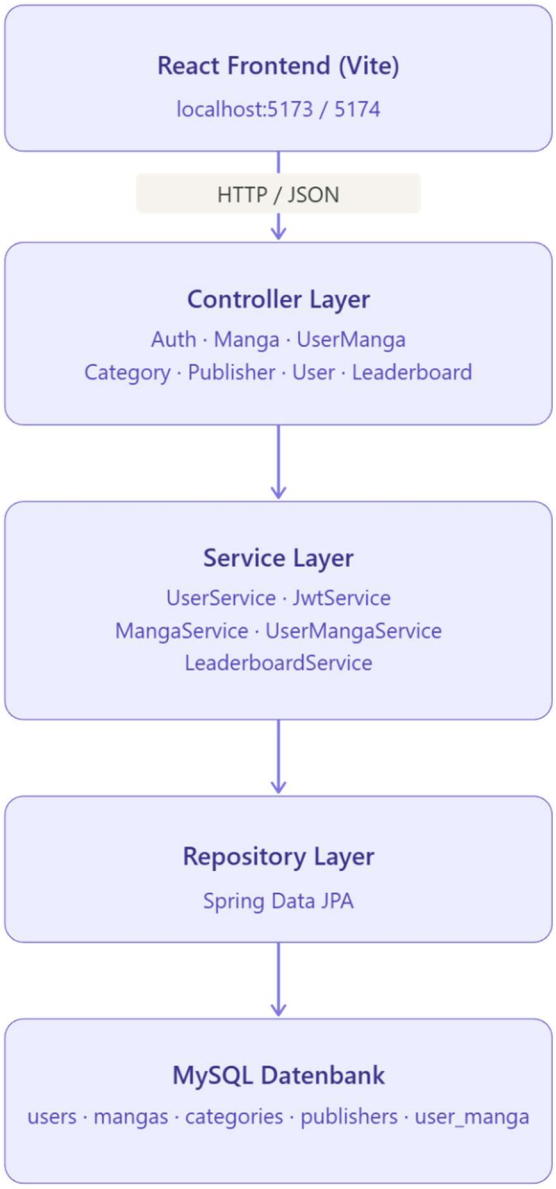
3.2 Backend-Architektur

Das Backend folgt einer klassischen Layer-Architektur. HTTP-Requests kommen zuerst bei den Controllern an. Dort werden sie validiert und an die Service-Schicht weitergegeben. Die Service-Schicht enthält Business-Logik, Transaktionen und Rollenregeln. Für den eigentlichen Datenbankzugriff werden Repositories verwendet. Die Daten werden in einer MySQL-Datenbank gespeichert.

Wichtige Backend-Bausteine:

- Controller-Layer: AuthController, MangaController, CategoryController, PublisherController, UserController, UserMangaController, LeaderboardController
- Service-Layer: UserService, JwtService, MangaService, UserMangaService, LeaderboardService, UserDetailsServiceImpl
- Repository-Layer: UserRepository, MangaRepository, CategoryRepository, PublisherRepository, UserMangaRepository
- Security-Layer: JwtAuthenticationFilter
- Config: WebConfig,SecurityConfig

3.3 Architekturdiagramm



3.4 Wichtige Endpunkte

Methode	Endpoint	Beschreibung	Zugriff
POST	/auth/register	Benutzer registrieren	offen
POST	/auth/login	Login mit Username oder E-Mail	offen
GET	/mangas	alle Mangas laden	USER, ADMIN
POST	/mangas	Manga anlegen	ADMIN
PUT	/mangas/{id}	Manga bearbeiten	ADMIN
DELETE	/mangas/{id}	Manga löschen	ADMIN
GET	/user-manga	Statusliste laden	USER eigene, ADMIN alle
POST	/user-manga	Status anlegen	USER eigene, ADMIN alle
PUT	/user-manga/{id}	Status bearbeiten	USER eigene, ADMIN alle
DELETE	/user-manga/{id}	Status löschen	USER eigene, ADMIN alle
GET	/leaderboard	Ranking aller Benutzer	USER, ADMIN
GET	/leaderboard/user/{id}/stats	Detailstatistik eines Benutzers	USER, ADMIN

3.5 Transaktionen im Backend

Im Projekt werden Transaktionen sinnvoll in der Service-Schicht eingesetzt. Dadurch bleibt die Business-Logik zentral und die Datenbank bleibt konsistent und

Beispiele:

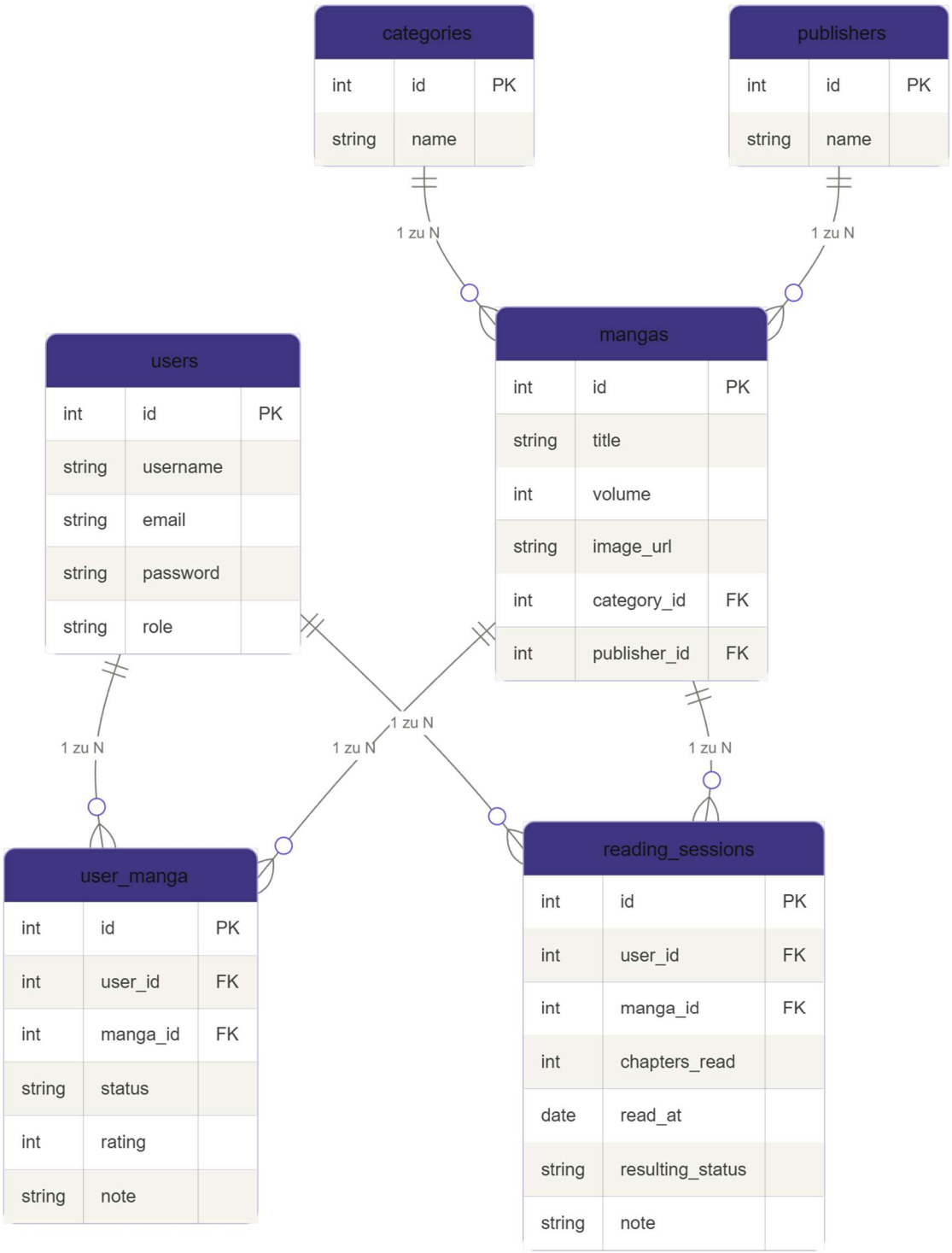
- MangaService kapselt Erstellen, Aktualisieren und Löschen von Manga-Einträgen.
- UserMangaService kümmert sich transaktional um die Erstellung, Änderung und Löschung von Benutzerstatus um Dateninkonsistenz zu vermeiden
- LeaderboardService arbeitet mit @Transactional(readOnly = true), weil dort nur gelesen und aggregiert wird und die durchschnittlichen Anzahl Manga User mit einer AVG Aggregate Funktion das durchschnittliche Rating berechnet wird

Warum ist das wichtig?

- Mehrere zusammengehörige Datenbankoperationen werden sauber in einer Einheit verarbeitet.
- Bei Fehlern entsteht kein halbfertiger Zustand sondern entweder eine oder keine Transaktion(ACID-Prinzip) ansonsten direkt Rollback
- Controller bleiben schlank und enthalten keine eigentliche Geschäftslogik.

4. Datenmodell und Diagramme

4.1 ERD-Diagramm



4.2 Beschreibung der Beziehungen

- Ein User kann viele user_manga Einträge besitzen.
- Ein user_manga-Eintrag gehört immer genau zu einem User.
- Ein user_manga-Eintrag verweist immer auf einen Manga.
- Ein Manga gehört genau zu einer Kategorie.
- Ein Manga gehört genau zu einem Verlag.
- Über die Unique-Constraint auf user_id und manga_id ist gesichert, dass ein Benutzer denselben Mangas nicht doppelt in seiner Statusliste führen kann

4.3 Seed-Daten

Die Datenbank wird beim ersten Start des MySQL-Containers mit einem Init-Skript vorbereitet. Dort werden Kategorien, Verlage, Benutzer, Mangas und erste Status-Einträge angelegt.

Beispielhafte Seed-Daten:

- Benutzer: admin, hans, tom, marco
- Kategorien: Shonen, Seinen, Shojo, Josei, Psychological, Sports
- Mangas: One Piece, Naruto, Attack on Titan, Bleach, Death Note, Blue Lock usw.

5. JWT-Authentifizierung und Sicherheitskonzept

5.1 Sicherheitsziel

Die Anwendung soll nur authentifizierten Benutzern Zugriff auf die geschützten Inhalte (Protected Routes) geben und handelt nach dem Zero Trust Prinzip. Zusätzlich soll zwischen normalen Benutzern und Administratoren unterschieden werden und nur berechtigte Admin User dürfen beispielsweise neue Mangas hinzufügen, bearbeiten oder neue User hinzufügen.

5.2 Rollenmodell

Es gibt zwei Rollen:

- USER
- ADMIN

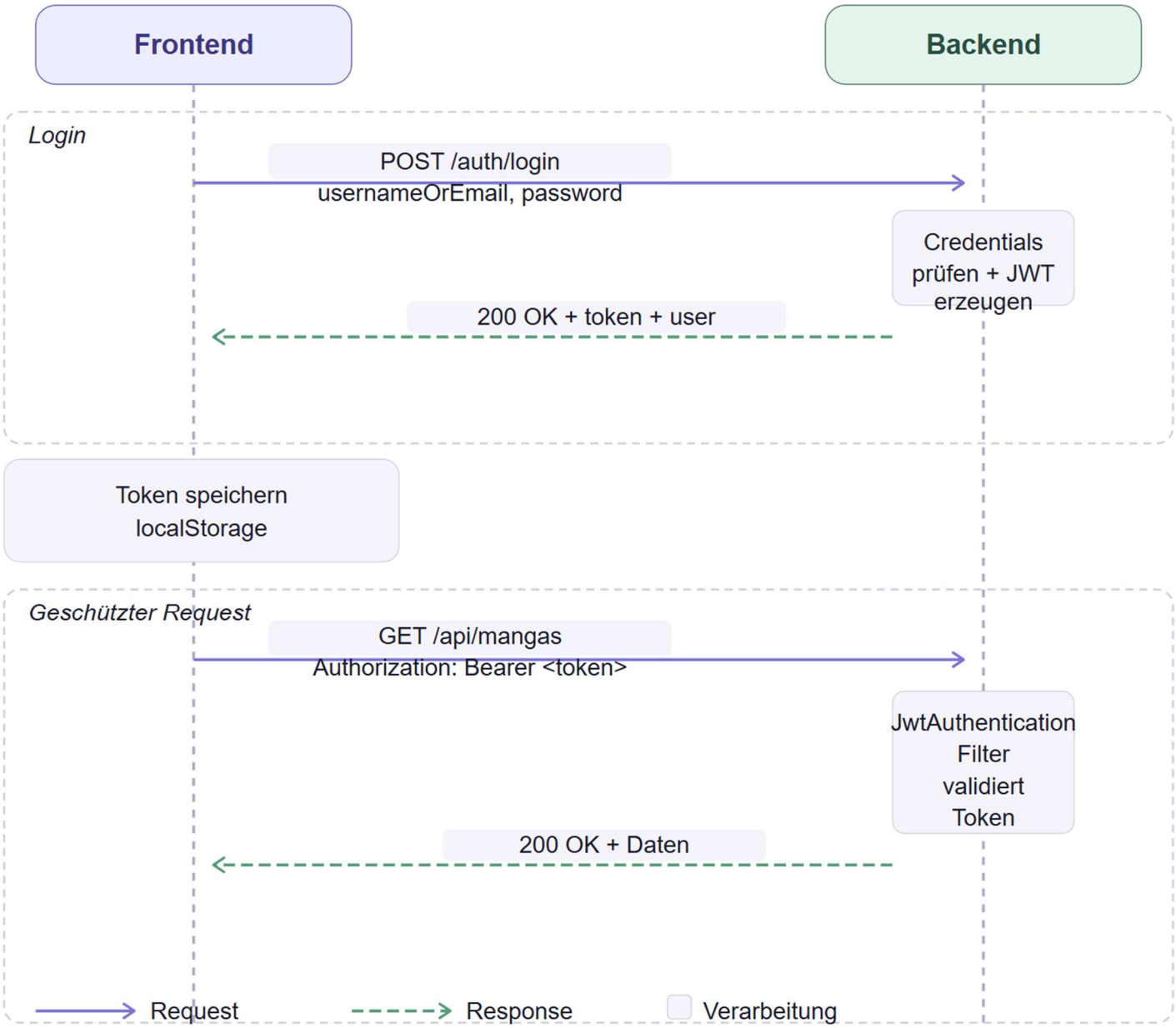
Bedeutung:

- USER darf lesen und eigene Manga-Status verwalten.
- ADMIN darf zusätzlich Stammdaten und Benutzer verwalten.

5.3 JWT-Flow

1. Benutzer sendet Login-Request an **/auth/login**
2. Backend prüft Username/E-Mail und Passwort
3. Bei Erfolg erzeugt JwtService einen Token
4. Frontend speichert den Token im localStorage
5. Axios sendet den Token bei späteren Requests im Authorization-Header
6. JwtAuthenticationFilter prüft den Token bei jedem geschützten Request und extrahiert Token und User
7. Bei gültigem Token wird der Benutzer in den SecurityContext geladen

5.4 JWT-Ablaufdiagramm



5.5 Schutzmechanismen

- Passwörter werden nicht im Klartext gespeichert, sondern mit BCrypt gehasht.
- Das Backend arbeitet stateless, also ohne serverseitige Session.
- SecurityConfig erlaubt nur /auth/** ohne Anmeldung und permitted die restlichen Pages und überprüft ob der User eingeloggt ist(isAuthenticated)
- MySQL Passwörter sind in YML-File in Environement Variabeln gespeichert
- Alle anderen Endpunkte sind standardmäßig authentifiziert zugänglich
- ProtectedRoute schützt die Frontend-Seiten.
- Rollenprüfungen erfolgen im Backend über Spring Security und @PreAuthorize und sowie dem JwtAuthentificationFilter

5.6 Besitzlogik bei UserManga

Ein besonders wichtiger Teil des Sicherheitskonzepts ist die Besitzlogik für Status-Einträge:

- Ein normaler Benutzer sieht nur die eigenen user_manga-Einträge.
- Ein normaler Benutzer darf nur eigene Einträge anlegen, bearbeiten und löschen.
- Ein Admin darf alle Einträge sehen und verwalten.
- Wird bei einem normalen Benutzer eine fremde userId mitgesendet, wird der Zugriff verweigert.

Das ist fachlich sinnvoll, weil Lesestatus persönlich ist und nicht von anderen Benutzern verändert werden soll.

5.7 CORS und Frontend-Integration

- Das Frontend läuft lokal über Vite.
- Das Backend lauft auf Port 8081.
- Im Frontend wird über den Vite-Proxy /api auf das Backend weitergeleitet.
- Standardmässig versucht Vite Port 5173, und hat vom Backend per @CrossOrigin vom Port 8081 zum Frontend und öffnet ausdrücklich, um HTTP-Request zu übermitteln➔Cross-Origin-Policy

6. Frontend-Dokumentation

6.1 Verwendete Technologien

Technologie	Verwendung
React 19	UI-Komponenten und Seiten
React Router DOM	Routing und geschützte Routen
Axios	HTTP-Kommunikation mit dem Backend
Vite	Entwicklungsserver und Build
Context API	globaler Auth-Zustand für lokaler Zugriff aus User Daten
Node Test Runner	kleine Frontend-Tests für Utilities

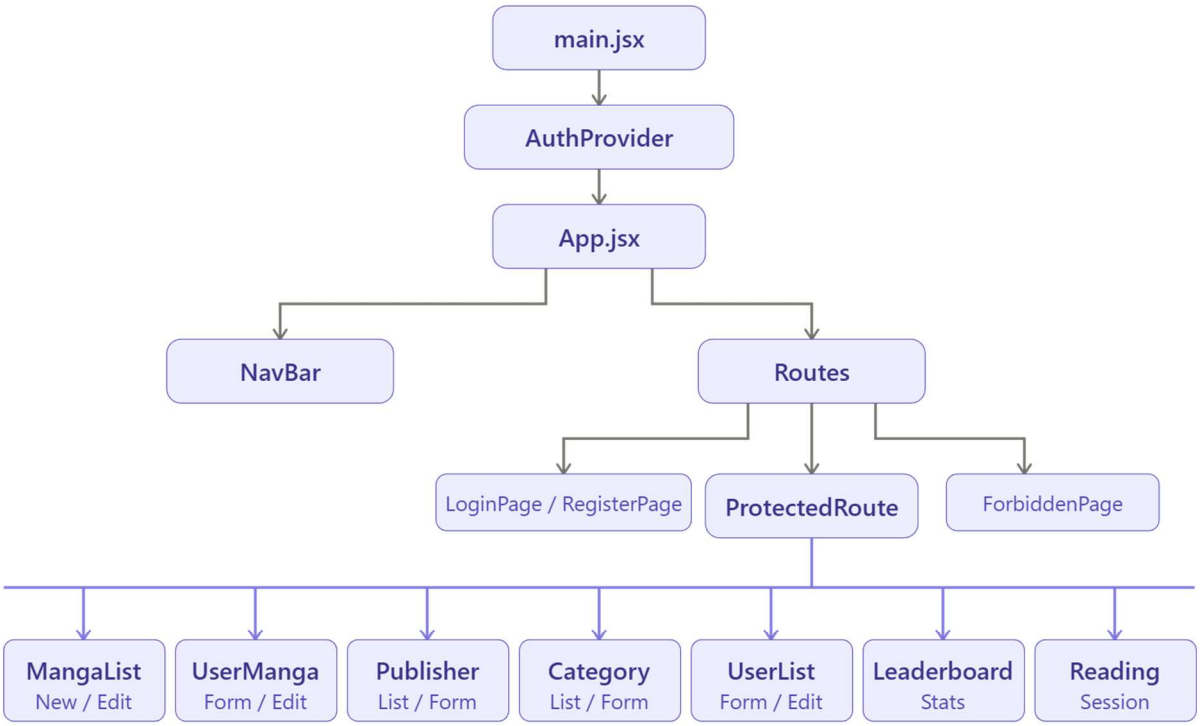
6.2 Frontend-Architektur

Das Frontend ist in Seiten, Komponenten, API-Module, Context und Utilities gegliedert. Die Anwendung startet in `main.jsx`, bindet den `AuthProvider` ein und rendert anschliessend `App.jsx`. In App.jsx werden alle Routes definiert. Geschützte Seiten werden mit ProtectedRoute abgesichert.

Wichtige Bereiche:

- Auth-Seiten: Login, Register, Forbidden
- Stammdaten für Manga: Mangas, Kategorien, Verlage, Benutzer
- Statusbereich: UserManga mit Enum für Manga (PLANNED READING, COMPLETED oder DROPPED)
- Leaderboard: LeaderboardPage mit Ranking der beliebtesten «Mangalesern»und Anzeige der geplanten zu lesenden Mangas
- Navigation: rollensensitive Menüpunkte in NavBar wie Admin und ProtectedRoutes

6.3 Komponenten-Hierarchie



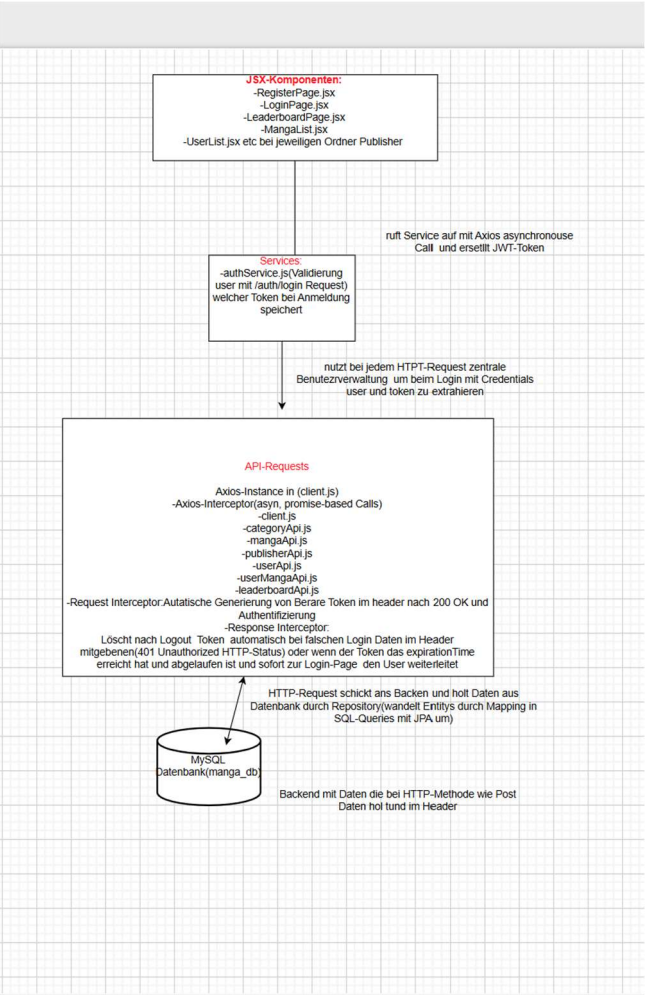
6.4 State-Management und AuthContext

Der wichtigste globale Zustand ist der Auth-Status. Der AuthContext verwaltet:

- user
- isAuthenticated
- isLoading
- login()
- logout()

Beim Laden der App wird geprüft, ob bereits ein Token und Benutzerdaten im Storage vorhanden sind. Ist das der Fall, bleibt der Benutzer eingeloggt.

6.5 API-Integration(Service Layer)



Das Frontend trennt die HTTP-Kommunikation sauber in API-Module:

- authService.js
- mangaApi.js
- userMangaApi.js
- publisherApi.js
- categoryApi.js
- userApi.js
- leaderboardApi.js

Diese Module nutzen zentral einen Axios-Client. Dadurch ist die API-Nutzung in den Seiten klar getrennt von der eigentlichen UI-Logik.

6.6 Rollenabhangige Darstellung im Frontend

- Nicht eingeloggte Benutzer sehen nur Login und Register.
- Eingeloggte Benutzer sehen Mangas, Leaderboard, Status, Verlage, Kategorien.
- Nur ADMIN sieht zusätzlich Users oder kann Manga hinzufügen
- In Listenansichten werden Admin-Aktionen nur dann angezeigt, wenn die Rolle sie wirklich ausführen darf.
- Ausgeloggte User werden direkt zur Login-Page redirected

7. Reading Sessions

7.1 Idee

Die ReadingSession-Funktion ergänzt die Manga-Anwendung um einzelne Leseereignisse. Dadurch wird nicht nur ein statischer Status gespeichert, sondern zusätzlich festgehalten, wann ein Benutzer einen bestimmten Manga-Band gelesen oder weitergeführt hat. Die Anwendung erhält damit einen fachlichen Mehrwert, weil Lesefortschritt als eigener Vorgang modelliert wird.

7.2 Funktionslogik

- Eine ReadingSession gehört immer zu genau einem Benutzer und genau einem Manga-Band.
- Beim Erstellen einer Session wird automatisch geprüft, ob bereits ein UserManga-Eintrag für diesen Benutzer und diesen Manga existiert.
- Falls noch kein Eintrag existiert, wird einer neu erstellt.
- Falls bereits ein Eintrag vorhanden ist, wird der Status sinnvoll angepasst.
- Wird der höchste bekannte Band eines Titels gelesen, wird der Status auf COMPLETED gesetzt.

7.3 Nutzen

- realistischerer Lesefortschritt
- gutes Beispiel für Transaktionen mit @Transactional
- sinnvolle Erweiterung der Manga-Domain

8. Leaderboard

8.1 Idee

Das Leaderboard rankt alle Benutzer danach, wie viele Mangas sie mit Status COMPLETED abgeschlossen haben. Damit bekommt die Anwendung einen kleinen statistischen und spielerischen und kompetitiven Mehrwert, ohne die eigentliche Manga-Domain zu verlassen.

8.2 Bewertungslogik

- Gewertet wird ausschließlich **COMPLETED** als Status.
- Bei Gleichstand entscheidet der Username alphabetisch.sss

- Zusätzlich werden weitere Werte angezeigt: readingCount, plannedCount, droppedCount

8.3 Leaderboard-Endpunkte

- GET /leaderboard
- GET /leaderboard/user/{id}/stats

8.4 Nutzen

- schneller Vergleich zwischen Benutzern a
- motivierender Überblick über den eigenen Lesefortschritt wie als Lesechallenge kompetitives Element
- gutes Beispiel für Aggregation und Reporting auf Datenbankebene mit Query

9. Testdokumentation

9.1 Backend-Tests

Test 1: AuthControllerIntegrationTest

Ziel: Prüfen, ob Registrierung und Login über die REST-API korrekt funktionieren.

Gepruft wird:

- erfolgreicher Register-Request
- erfolgreicher Login
- Fehlerfall bei ungültigen Login-Daten

Ergebnis: Die Authentifizierung funktioniert über die API wie erwartet und liefert beim Login einen JWT-Token mit Benutzerdaten zurück.

Test 2: UserMangaControllerSecurityTest

Ziel: Prufen, ob normale Benutzer nur ihre eigenen Status-Einträge verwalten können.

Gepruft wird:

- Benutzer sieht nur eigene Einträge
- Benutzer kann keinen fremden Eintrag verändern
- Admin hat erweiterten Zugriff

Ergebnis: Die Besitzlogik für user_manga funktioniert und verhindert unberechtigte Zugriffe.

Test 3: LeaderboardControllerIntegrationTest

Ziel: Prüfen, ob das Leaderboard korrekt aggregiert und geschützt ist.

Gepruft wird:

- nicht authentifizierte Requests werden blockiert
- alle Benutzer erscheinen im Ranking
- COMPLETED wird korrekt gezahlt
- Detailstatistik pro Benutzer funktioniert

Ergebnis: Das Ranking arbeitet korrekt und ist nur für angemeldete Benutzer erreichbar.

Test 4: MangaControllerSecurityTest

Ziel: Prüfen, ob das Leaderboard korrekt aggregiert und geschützt ist.

Geprüft wird:

- nicht authentifizierte Requests werden blockiert und erhalten eine Errormeldung(«Nicht authentifziriert»
- alle MangasmitderId1 ohne Authentifizierung mit Rolle(User/Admin) erhaltenError-Fehlermeldung 401)
- Login mit fiktivem User «user1 wird gemockt und gibt 403(Forbidden)tTu-Meldug mit error-Message(«Zugriff-verweiger»), da er auf Protected Routes kein Zugriff hat wie /mangas
- Authorisierter user «admin» hat Zugriff auf Mangaliste➔Status Meldung 20, User wird gemockt

Ergebnis: Das Ranking arbeitet korrekt und ist nur für angemeldete Benutzer erreichbar.

9.2 Frontend-Tests

Test 1: authUtils.test.js

Ziel: Prüfen, ob Token und Benutzerdaten korrekt gespeichert und gelesen werden.

Gepruft wird:

- Auth-Session speichern
- Token lesen
- User lesen
- Storage bereinigen

Ergebnis: Die Grundlage für den Login-Zustand im Frontend ist verifiziert.

Test 2: mangaUtils.test.js

Ziel: Prüfen, ob Hilfsfunktionen für die Manga-Anzeige korrekt arbeiten.

Geprüft wird:

- Gruppierung und Aufbereitung von Manga-Daten
- stabile Darstellung für die Liste

Ergebnis: Die Datenaufbereitung für die Manga-Seite ist reproduzierbar und getestet.

Test 3: leaderboardUtils.test.js

Ziel: Prüfen,

ob Werte im Leaderboard korrekt formatiert werden.

Gepruft wird:

- fehlende Durchschnittsbewertung wird als - angezeigt
- vorhandene Werte werden sinnvoll gerundet

Ergebnis: Die Anzeige im Frontend bleibt auch bei unvollstandigen Statistikdaten stabil.

9.3 Testprotokoll

Verwendete Befehle:

```
cd manga-backend
mvn test oder mvn run test
```

```
cd manga-frontend
npm.cmd run build
```

```
cd manga-frontend
```

```
node --test src/pages/Leaderboard/leaderboardUtils.test.js
src/pages/Mangas/mangaUtils.test.js src/utils/authUtils.test.js
```

10. Git und Entwicklungsprozess

10.1 Verwendete Remote Branches

- feature/backend-security
- feature/frontend-auth
- feature/leaderboard
- feature/unit-tests
- unit-tests

10.2 Nutzen der Branch-Strategie

Die Features wurden nicht direkt auf main umgesetzt, sondern zunachst in eigenen Branches entwickelt. Das war sinnvoll, weil:

- Backend und Frontend getrennt entwickelt werden konnten
- Features als Pull Request nachvollziehbar blieben
- Commit-Historie besser lesbar blieb
- Risiken beim Integrieren reduziert wurden

10.3 Beispielhafte Commit-Gruppen

- Backend-Security und JWT
- Frontend-Authentifizierung mit AuthContext und ProtectedRoute
- Zugriffsbeschränkungen für UserManga
- Leaderboard für gelesene Mangas
- Seed-Daten mit neuen Kategorien und Cover-Links

10.4 Warum Git für die LB wichtig ist

Git ist in diesem Projekt nicht nur ein Speichertool, sondern Teil der Bewertung. Durch Branches beispielsweise für die Security eine einzelne und klar benannte Commits ist sichtbar, wie das Projekt Schritt für Schritt aufgebaut wurde. In der Main Branch befinden sich die Commits mit der lauffähigen Webapplikation, im uni-test Branch alle Unit-Tests sowie im security alle Files mit der Spring Security Implementation du im leaderboard Feature die User-Stats zu den beliebtesten Mangas angezeigt werden.

11. Start und Betrieb des Projekts

11.1 Datenbank starten

```
cd 223_Projekt_fresh\manga-backend
docker compose up -d
```

11.2 Backend starten

```
cd 223_Projekt_fresh\manga-backend
mvn spring-boot:run oder docker compose up -build oder manuell  mit grünem
Play Button in der Main Klasse(MangaApiApplication)
```

Backend: `http://localhost:8081`

11.3 Frontend starten

```
cd 223_Projekt_fresh\manga-frontend
npm.cmd
 Command um Vite-Server zu starten: npm run dev
Frontend Depencies installiern fürs Deployement des Viteservers
-npm install
-npm install axios
```

Standard: `http://localhost:5173`

Falls belegt: `http://localhost:5174  oder anderer direkt von Vite geöffnet und gestartet`

11.4 Test-Logins

Admin: `admin / admin123`

User: `hans / welcome123`